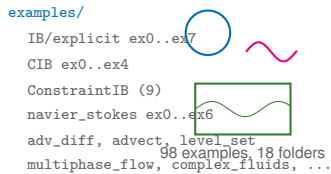
TOUR OF THE UPSTREAM EXAMPLES

What the examples that ship with IBAMR do: the IB, CIB, ConstraintIB, Navier–Stokes, advection–diffusion, level-set, multiphase, complex-fluid and other suites, how to build them, and which ones ran



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 15

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. find your way round the `examples/` folder of the IBAMR source tree: the eighteen suites, what physics and which solver class each one exercises, and which of this series' chapters goes with it;
2. say what each of the eight basic IB examples does, which files it needs, how long it runs, and what to look at in VisIt; the same for the CIB and Navier–Stokes suites, and in outline for the rest;
3. build the examples two ways: inside IBAMR's own build tree with `make examples-IB` or `make IB-ex1`, and outside it, against an installed (autoibamr) IBAMR, with the plain `CMakeLists.txt` of this series;
4. map the example names of the 2013 and 2015 course decks onto today's folders, and know which three statements of the older decks were wrong;
5. pick the example nearest to your own problem as a starting point, and know what it will cost to run.

> Everything in chapters 5–14 was built on these examples. This chapter is the map: forty of them were built and started for it on IBAMR 0.19.0, and the table on slides 34 and 35 says what happened.

HOW THIS CHAPTER WAS CHECKED

- The example list, the files, the input parameters, the README texts and the classes each program creates are read from the IBAMR 0.19.0 source tree (`examples/`), not from memory or from the older decks.
- Forty of the examples (every 2D one of the IB, CIB, Navier–Stokes, advection–diffusion, level-set and complex-fluid suites, one or two of each other suite, and the 3D ones that are cheap) were built against the installed 0.19.0 with a plain `CMakeLists.txt` and run for up to two minutes on two ranks; slides 34 and 35 report every build and run, including the ones that did not work that way. The gallery pictures are VisIt renders of those runs.
- The 2020 lecture *What do Boyce's examples do?* (IBAMR Lecture 6, 25 slides), the 2013 *IBAMR Examples* deck (18 slides; re-uploaded unchanged as the 2015 tutorial "Boyce's Examples") and the 2012 lab ReadMe's "Other examples" slide are quoted in full; where a slide was wrong about the code, it is quoted and corrected.
- The IBFE examples are listed here and toured in chapter 25; ConstraintIB's nine were chapter 14's.

> Slide numbers refer to this deck; chapter numbers (2, 3, 5, 6, 7, 9a, 10, 11, 12, 13, 14, 16, 24, 25, 26) to the other chapters of this series.

WHERE TO LOOK THINGS UP

- The tree: `<autoibamr>/tmp/unpack/IBAMR-0.19.0/examples/` (chapter 2), or https://github.com/IBAMR/IBAMR/tree/master/examples.
- Each example: its `example.cpp` (the program; `main.cpp` in a few), `input2d`/`input3d` and their variants, the structure files, sometimes a `README`, a MATLAB generator or an analysis script.
- The build: `examples/CMakeLists.txt` (the `IBAMR_ADD_EXAMPLE` macro, slide 9) and each suite's `CMakeLists.txt` for the target names.
- The papers: each README names its paper where there is one; the lectures named Bhalla et al. (2013) for ConstraintIB, Kallemov et al. (2016) for CIB, Barrett's dissertation for complex fluids, Griffith & Lim (2012) for the rod, Zhu & Peskin (2002) and Kim & Peskin (2007) for the filament.
- The lectures: IBAMR Lecture 6 (2020), the 2013/2015 examples deck, the 2012 ReadMe (chapter 0.1's source list).

THE EXAMPLE TREE IN IBAMR 0.19.0 (1): STRUCTURES

folder	count	what it exercises	chapters
`IB/explicit`	8	the standard IB method: bands, shells, a cylinder, a filament with mass, a rod, random points, a minimal integrator, posts	5–13, this one
`CIB`	5	rigid bodies by the constraint method with mobility solvers (Stokes)	this one
`ConstraintIB`	9	rigid and prescribed-deformation bodies by velocity correction	14
`IBFE/explicit`	12	the finite-element IB method	25
`IMP/explicit`	1	the immersed material point method	this one
`IIM`	10	the immersed interface and immersed Lagrangian–Eulerian methods (libMesh surfaces)	this one
`DLM`	1	distributed Lagrange multipliers: the Turek–Hron benchmark	this one
`IBLevelSet`	7	fully Eulerian FSI with level sets	this one

98 examples in 18 suites (the eighteenth, `IB`, holds only `explicit`). The solid-mechanics suites first; the fluid-only and multiphase suites follow on the next slide.

THE EXAMPLE TREE IN IBAMR 0.19.0 (2): FLUIDS, SCALARS, FREE SURFACES

folder	count	what it exercises	chapters
`navier_stokes`	7	the fluid solver alone: exact solutions, cavities, Boussinesq convection	this one, 16
`vc_navier_stokes`	4	variable-density, variable-viscosity solver checks	this one
`adv_diff`	9	a scalar advected and diffused; flux-based penalization for Neumann/Robin walls	this one, 24
`advect`	1	pure advection of a blob	this one
`level_set`	2	level-set initialization and transport	this one
`multiphase_flow`	14	air–water flows: drops, dam breaks, bodies hitting a free surface	this one
`wave_tank`	2	numerical wave tanks	this one
`complex_fluids`	5	Oldroyd-B fluids	this one
`fe_mechanics`	1	solid mechanics alone (Cook's membrane)	this one

98 examples. `examples/CMakeLists.txt` lists the suites in the order `CIB ConstraintIB DLM IB IBFE IBLevelSet IMP IIM adv_diff advect complex_fluids fe_mechanics level_set multiphase_flow navier_stokes vc_navier_stokes wave_tank`; the 2020 lecture's slide 2 named five of them, "IB – basic immersed boundary examples, IBFE – Hybrid IB/Finite element examples, Constraint IB – Bhalla et al., 2013, CIB – Kallemov et al., 2016, Complex fluids – Barrett dissertation", with a screenshot of the folder listing.

THREE GENERATIONS OF “BOYCE’S EXAMPLES” IN THE COURSE

- 2012, the lab ReadMe, slide 10 “Other examples”: “There are many other examples that Boyce has put together in the examples folder. You can go into each folder, make the examples, and run them. IB are immersed boundary examples, IBFE are immersed boundary – finite element, adv_diff include advection diffusion, etc. For example, do `cd IB`, `cd staggered_grid`, `cd explicit`, `make examples`, `cd ex0`, `make main2d`, `./main2d input2c`” (the last is a typo for `input2d`; the path `IB/staggered_grid/explicit` is the 2012 layout, today `IB/explicit`).
- 2013, *IBAMR Examples* (MATH 769, April 4, 2013; 18 slides), re-uploaded in 2015 as tutorial 12 “What do Boyce’s examples do?”: one slide each for `IB/explicit` ex1–ex4 (the old numbering, slide 32), `navier_stokes` ex0, ex2, ex4, ex5, `advect` and `adv_diff/ex0`, with VisIt screenshots and input excerpts.
- 2020, IBAMR Lecture 6 *What do Boyce’s examples do?* (25 slides): how to run them on the UA cluster (slides 3–5), `IB/explicit` ex0–ex5, `CIB` ex0–ex4, ConstraintIB (six slides, chapter 14), and “More examples next week: For next week, I’d like to go through the non-Newtonian and level set examples. Then I’d like to revisit the IBFE ones. I encourage you to run and look at examples that are related to your target applications and discuss them next week.”

> The suite grew from about 25 examples in 2013 to 98 in 0.19.0; this chapter covers all 98 in outline and the ones the lectures opened in detail.

BUILDING THEM IN 2020: COPY, MAKEFILE, SCRIPT

"Running Boyce's examples. You will want to copy the examples folder to your xdisk space. `$ pwd /groups/lauram9/ib16/Code/IBAMR $ cp -r examples /xdisk/lauram9/lauram9/examples`. You will also want to move a Makefile and a script.sh into each example folder you plan to run. Text for the Makefile is on the next slide and also on github." (2020, slide 3)

```
# slide 4, "Makefile text" (the parts that matter)
IBAMR_SRC_DIR = /groups/lauram9/ib16/Code/IBAMR
IBAMR_BUILD_DIR = /groups/lauram9/ib16/Code/ibamr-objs-opt
LIBSNEW = $(LIBS) -lnetcdf -lcurl
include $(IBAMR_BUILD_DIR)/config/make.inc
SOURCES = example.cpp
OBJS = example.o
main2d:
        if (test -f stamp-3d); then $(MAKE) clean; fi
        touch stamp-2d
        $(MAKE) PDIM=2 main-2d
main-2d: $(IBAMR_LIB_2D) $(IBTK_LIB_2D) $(OBJS) $(SOURCES)
        $(CXX) $(CXXFLAGS) $(LDFLAGS) $(OBJS) \
        $(IBAMR_LIB_2D) $(IBTK_LIB_2D) $(LIBSNEW) -DNDIM=$(PDIM) -
  o main2d
```

```
# slide 5, "Sample script text"
#!/bin/bash
#SBATCH --job-name=CIB_Examples
#SBATCH --ntasks-per-node=16
#SBATCH --nodes=1
#SBATCH --time=1:00:00
#SBATCH --partition=standard
#SBATCH --account=lauram9
#SBATCH --output=IBAMR2D.output

mpirun ./main2d input2d
```

That Makefile is the autotools era (chapter 3): it includes `make.inc` from an IBAMR *build* directory and links the 2D or 3D library by hand; "Be sure to change all of the 2's in the Makefile to 3's" (slides 7, 8) for a 3D example. The commented-out paths in it point at the 2018 dogwood module at UNC. Both the source tree and the build directory it names are gone; chapter 2's autoibamr puts the sources in `<autoibamr>/tmp/unpack/IBAMR-0.19.0` and the library in `<autoibamr>/packages/`.

BUILDING THEM NOW (1): INSIDE IBAMR'S OWN BUILD TREE

```
# examples/CMakeLists.txt (IBAMR 0.19.0), the helper every suite
  uses
# IBAMR_ADD_EXAMPLE(
#   TARGET_NAME       "IBFE-ex0"
#   OUTPUT_DIRECTORY  "${CMAKE_BINARY_DIR}/examples/IBFE/explicit/
  ex0"
#   OUTPUT_NAME       main2d
#   EXAMPLE_GROUP     examples-IBFE
#   SOURCES           "example.cpp"
#   REQUIRES          IBAMR_HAVE_LIBMESH
#   LINK_TARGETS      IBAMR2d
#   INPUT_FILES       input2d cylinder2d.vertex petsc_options.dat
#   )
ADD_CUSTOM_TARGET(examples)
SET(EXAMPLE_DIRECTORIES CIB ConstraintIB DLM IB IBFE IBLevelSet IMP
  IIM adv_diff
  advect complex_fluids fe_mechanics level_set multiphase_flow
  navier_stokes
  vc_navier_stokes wave_tank)
    ADD_EXECUTABLE("${EX_TARGET_NAME}" EXCLUDE_FROM_ALL ${EX_SOURCES
  })
    FOREACH(_input_file ${EX_INPUT_FILES})
      CONFIGURE_FILE("${_input_file}" "${EX_OUTPUT_DIRECTORY}"
  COPYONLY)
    ENDFOREACH()
  ELSE()
    MESSAGE(STATUS "Example ${EX_TARGET_NAME} requires libMesh -
  skipping")
```

In the directory where IBAMR itself was configured with CMake (autoibamr's `tmp/build/v0.19.0` while it exists, or your own):

```
make examples          # every example (EXCLUDE_FROM_ALL:
  not built by default)
make examples-IB       # one suite
make IB-ex1            # one example
cd examples/IB/explicit/ex1 && mpirun -np 4 ./main2d input2d
```

The executable is named `main2d` or `main3d` and placed in `<build>/examples/<suite>/<ex>/` together with the `INPUT_FILES` and `EXTRA_FILES` the suite's `CMakeLists.txt` lists (inputs, vertex files, `petsc_options.dat`, MATLAB scripts), so you run there, not in the source tree. Examples whose `REQUIRES` is not met (libMesh for IBFE, IIM, parts of multiphase and complex fluids) are skipped with the message shown. Chapter 2 timed `make -j4 IB-ex1` at 28 s.

BUILDING THEM NOW (2): OUTSIDE THE TREE, AGAINST AN INSTALLED IBAMR

```
# CMakeLists.txt dropped into a copy of any example folder (this
  series' form, chapter 3)
cmake_minimum_required(VERSION 3.15...3.30)
project(tour LANGUAGES C CXX)
find_package(IBAMR 0.16.0 REQUIRED)
add_executable(main2d example.cpp)          # plus the example's
  other .cpp files
target_link_libraries(main2d PRIVATE IBAMR::IBAMR2d)
```

```
cp -r $IBAMR_SRC/examples/IB/explicit/ex1 ~/work/ex1 && cd ~/work/
  ex1
# write the CMakeLists.txt above (replace the suite's own, which
  needs the macro)
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 4
mpirun -np 4 ./build/main2d input2d
```

This is how the forty examples of slides 34 and 35 were built: copy the folder, replace its `CMakeLists.txt` (the upstream one calls `IBAMR_ADD_EXAMPLE` and links `IBAMR2d`, names that exist only inside IBAMR's build), list every `.cpp` of the folder except helper programs, and link `IBAMR::IBAMR2d` or `IBAMR::IBAMR3d`. autoibamr deletes its build tree after installing unless told to keep it, so on a machine set up by chapter 2 this route is usually gone; this route always works. What to leave out and where to run: next slide.

BUILDING THEM NOW (3): WHAT TO LEAVE OUT, AND WHERE TO RUN

- Helper programs are separate executables upstream: `convergence_tester.cpp` (`navier_stokes/ex1`, `complex_fluids/ex2`), the `*Gen.cpp` vertex generators of `multiphase_flow` (“This file is automatically generated by the build process”), `write_ascii.cpp` (`adv_diff/ex8`). Leave them out of `main2d`, or build each as its own target.
- Inputs with suffixes (`input2d.cylinder`, `input2d.plate`, `input2d_CaseA`) are passed by name.
- `petsc_options.dat` (CIB) is read from the working directory.
- The 3D examples need `main3d` and `IBAMR::IBAMR3d`; the same `example.cpp` compiles for both (`NDIM` comes from the library target).

The same six lines serve every suite; the forty builds of slides 34 and 35 were made by a script that writes them, lists the folder’s `.cpp` files minus the helpers, and runs `cmake` (`wsl/final/run_tour.sh` in this series’ working files).

IB/EXPLICIT (1): EX0, THE PRESSURIZED BAND MADE IN CODE

ex0, a pressurized band. “This is an example of a pressurized rubber band. The figure shows a color map of the pressure and the resulting velocity vectors. Input2d was used in this simulation. There is also input2d.shell and input2d.shell_circum_fibers that you can use to make different kinds of rubber bands.” (2020, slide 6)

- no `.vertex` file: the band is generated in `example.cpp` by `IBRedundantInitializer`, which takes a function that returns the points and springs (the one initializer that needs no files; chapter 6 named the other);
- a 1 m periodic box, 64×64 cells, one level; $\mu = 10^{-2}$, $\rho = 1$, stiffness `K = 1`; the run is 0.55 s in $2{,}200$ steps (`DT = 1.6e-2*DX`), eight seconds on two ranks;
- `input2d.shell` and `.shell_circum_fibers`: a band of finite width `W = 0.0625` with radial, or radial and circumferential, springs.

Why it matters: every other example starts from files, and chapter 6 is about those files; ex0 shows the one way round them, which is also the way a generator written in C++ (chapter 12’s 3D cases) hands points to IBAMR.

IB/EXPLICIT (2): EX1, THE BOUNCING ELLIPSE AND THE SHELLS

ex1, the bouncing ellipse. “This is an example of a bouncing rubber band initialized as an ellipse. The figure shows a color map of the pressure and the resulting velocity vectors. Input2d was used in this simulation. As in the other example, there is input2d.shell and input2d.shell_circum_fibers. There is also a 3D shell you run with input3d. Be sure to change all of the 2’s in the Makefile to 3’s.” (slide 7)

- the same box and fluid as ex0, but the structure comes from files: `curve2d_64` to `_512` (`.vertex`, `.spring`) from `generate_curve2d.m`, the shells `shell2d_*` and `shell2d_radial_*` from `generate_shell2d.m`, and `sphere3d` for `input3d` (three levels, $\mu = 0.005, 2.4$ s);
- the 2013 deck’s “ex1 – Bouncing rubberband” and “ex1 – Bouncing shell” are this example: “a band that resists stretching. It has periodic boundary conditions” and “The structure is reinforced in the radial and circumferential directions. The shell starts as an ellipse and bounces around a circular shape. .spring and .vertex files are used to make the structure.”;
- this series’ IB/01 (chapters 3, 5, 6) is ex1’s band with its own files; chapter 16 uses ex1’s four resolutions for a convergence test.

IB/EXPLICIT (3): EX2, FLOW PAST A CYLINDER OR A SPHERE

"This is an example of flow past a cylinder. The example includes analysis files to calculate the Strouhal number. The figure shows a color map of the vorticity and the resulting velocity vectors. Input2d.cylinder was used in this simulation. There is also a 3D cylinder and a sphere with corresponding input files. Be sure to change all of the 2's in the Makefile to 3's." (2020, slide 8)

- `input2d.cylinder`: $R = 0.5$, $\mathrm{Re} = 200$ (`MU = 2R/Re`), five levels refined by 2 from $N = 16$; the inflow ramps up smoothly (next slide); `END_TIME = 300`: a Puma job (chapter 1), not a laptop run; two minutes on two ranks gave $9{,}649$ steps (slide 34);
- the tethering is in the input, not a `.target` file: `uniform_target_stiffness` and `uniform_target_damping` under the structure's block of `IBStandardInitializer` (chapter 9a's keys);
- `input2d.cylinder_stabilized`: $\mathrm{Re} = 5000$ with the stabilized convective operator;
- `input3d.cylinder`, `input3d.sphere` (and `.cylinder.test`, 0.1 s): a 16 m box, two levels, $\mathrm{Re} = 100$ and 200; vertex files from 32 to 1024 points across;
- `compute_St.m`: the Strouhal number from the lift history (chapter 11).

IB/EXPLICIT (3B): EX2 IN THE 2013 DECK, AND THE RAMPED INFLOW

2013, slides 4–5: “This was run with input2D, and it’s tethered in place. Other options include a 3D cylinder and sphere. 3D sphere and cylinder flow also available. Target stiffness and damping included. Done in a way that doesn’t use .target file. ‘Fancy’ boundary conditions in 2D. U1 has prescribed inflow that is a function of time, the other boundaries are Neumann. U2 is zero on top and bottom, is function of time and position on the inflow, and has Neumann outflow.”

```
// input2d.cylinder: the ramped inflow on the left side (chapter 7)
gcoef_function_0 = "(tanh((t-t_half)/tau)+tanh(t_half/tau))/(1+tanh(t_half/tau))"
```

The ramp is chapter 7’s time-dependent Dirichlet value: the inflow speed rises from 0 to 1 over about `tau` around `t_half`, so that the cylinder is not hit by an impulsive start (chapter 16’s first rule). The 2013 deck’s ‘fancy’ boundary conditions are these: a prescribed inflow on the left, zero normal velocity top and bottom, and Neumann outflow on the right.

IB/EXPLICIT (4): EX3, THE TETHERED FILAMENT WITH MASS

"This is an example of flow past a flexible, tethered filament. The figure shows a color map of the vorticity and the resulting velocity vectors. This example was inspired by a paper by Zhu and Peskin. Go to the google drive papers folder to find it." (2020, slide 9)

2013, slides 6–9: "Inspired from an experiment. Mass was necessary to induce flapping. This simulation should flap at higher Re or higher mass." "2D flow over a fiber with mass that resists bending and stretching and is tethered at the top. Boundary conditions allow flow in the y-direction. U1 is fixed on all sides to zero. U2 is set to Neumann B.C.'s. Flow is generated due to the effect of gravity. This comes from a 'forcing function' in the input file." "The boundary mass is added in an older way (no .mass files are used)." "Penalty IB method. This specific application is described in Zhu and Peskin. A '.mass' file is used to specify any additional mass associated with the IB points. For such files to have any effect, it is necessary that the IB solver be run in 'penalty-IB' mode."

```
// input2d (0.19.0): the lines the 2013 deck quoted, still there
ForcingFunction {
   function_0 = "0.0"
   function_1 = "-g*rho*( 1 + (F-1)*exp(-400*t) - F*exp(-600*t) )"
}
IBStandardInitializer {
   structure_names = "fila_256"
   fila_256 {
      level_number = MAX_LEVELS - 1
      ds = 3.84 / NFINEST
      uniform_spring_stiffness = spring_stiffness
      uniform_spring_rest_length = ds
      uniform_beam_bend_rigidity = bend_rigidity*ds/(ds^4)
      uniform_bdry_mass = bdry_mass_density*ds
      uniform_bdry_mass_stiffness = bdry_mass_stiffness
   }
}
```

`example.cpp` creates `PenaltyIBMethod` (chapter 13): the "older way" of 2013 is the `uniform_bdry_mass` keys, which give every point a massive copy without a `.mass` file. The fluid is a gas ($\rho = 3 \times 10^{-4}$, $\mu = 1.2 \times 10^{-3}$, "Re=210"), gravity 980 cm/s^2 ramps up through the forcing function, and five levels refined by 2 from $N = 16$ in a 9 cm box. `DT = 0.128/NFINEST^2` is 2×10^{-6} s: the 1.2 s run is $600{,}000$ steps, a cluster job; two minutes on a desktop PC gave slide 34's count. Files for 256, 512 and 1024 points.

IB/EXPLICIT (5): EX4, THE KIRCHHOFF ROD

ex4. “This is an 3D example of a rubber band that starts out as a circle that is loaded with torsion. The infinitely thin boundary can apply rotational forces to the fluid. Eventually the rubber band twists in 3D as seen in this snapshot. Griffith, Boyce E and Lim, Sookkyung. Simulating an Elastic Ring with Bend and Twist by an Adaptive Generalized Immersed Boundary Method. Communications in computational physics, 08/2012, Volume 12, Issue 2, pp. 433–461.” (2020, slide 10)

2013, slide 11: “IB/explicit/ex4 - Kirchoff Rods. 3D flow around a 1D ring that has an intrinsic curvature and twist. The fluid-rod interaction problem is solved by the generalized immersed boundary method combined with the Kirchhoff rod theory. The files include .rod and .director files.” (“Kirchoff” is the slide’s spelling; the links it gave, to math.nyu.edu and pof.aip.org, are dead.)

Files: `curve3d_64.vertex`, `.rod`, `.director` from `generate_mesh3d.m`; `example.cpp` uses `IBKirchhoffRodForceGen` in place of `IBStandardForceGen`. A 10 cm box, three levels refined by 2 to 64 cells, $\mu = 0.01$, 100 s of run: a 3D cluster job (slide 34 has the two-minute start).

IB/EXPLICIT (6): EX5, WHICH IS NOT WHAT THE LECTURE SAID

ex5, corrected. “This example includes a bunch of immersed boundary points that each move with their own behavior. The vertices and their x, y-coordinates are outputted, but the flow fields are not.” (2020, slide 11)

That is not what ex5 is. Its input begins with “random-number generator parameters” (`SEED`), a temperature `kT = 1.0`, and the fluid parameters; `example.cpp` creates `INSStaggeredStochasticForcing`: it is *fluctuating hydrodynamics*, a fluid with thermal noise in which a cloud of free IB points (`points2d.vertex`, `points3d.vertex`, with no springs) diffuses by Brownian motion. The points have no behaviour of their own; the fluid is random. The flow fields *are* written: `viz_writer = "VisIt","Silo"` with `VISIT_DUMP_INTERVAL = 10`, and the positions go to ASCII `X.*` files besides (chapter 11’s format). `DT = 10.0` and `END_TIME = N_STEPS*DT`: the run is counted in steps, not seconds.

What the lecturer saw, a scatter of dots and no flow, was VisIt showing the Lagrangian mesh alone (slide 11’s picture): the fluid data were there in `viz_IB2d`.

IB/EXPLICIT (7): EX6, A MINIMAL INTEGRATOR IN TWO FILES

ex6, a minimal IB integrator. The band of ex1 (`curve2d_64` to `_512`, `generate_curve2d.m`, the same input) run by `IBSimpleHierarchyIntegrator`, a time integrator written in the example's own two files (`IBSimpleHierarchyIntegrator.cpp`, `.h`) instead of the library's `IBExplicitHierarchyIntegrator`: forward Euler for the points, the standard spread and interpolate, and nothing else. It is the shortest complete statement of chapter 5's algorithm in IBAMR's own terms, about two hundred lines, and the place to start if you ever need an integrator of your own. Eight seconds for the $2{,}200$ steps.

Read `IBSimpleHierarchyIntegrator.cpp` next to chapter 5's slides: `integrateHierarchy` spreads the force, solves the fluid, interpolates the velocity and moves the points, in that order and nothing else.

IB/EXPLICIT (8): EX7, POSTS IN 3D WITH A SCALAR

ex7, posts in 3D with a scalar. README: “This is a tutorial example used to track the IB coordinates of a single post which undergoes prescribed motion. In this example, the IB coordinates of the post are prescribed to lie and move along the level set of a cone who’s radius and length is set by the post_deflection_radius and post_length either in the input file, or using the default values as specified in the example.cpp file. After running the executable associated with this example, a ‘IBCoordinates.txt’ file is generate whose columns correspond to the relevant Lagrangian index as specified in the .vertex file and the Eulerian x,y,z coordinates. . . . The MATLAB script ‘generate_posts.m’ is used to generate the IB points of the posts located at the center of the square domain. . . . Notice that a number of problem parameters are currently ‘hard coded’ in this script.”

A $1 \times 1 \times 0.25$ cm box of water, three levels, an advected–diffused scalar (`D = 1e-3`, `AdvDiffSemiImplicitHierarchyIntegrator` alongside the IB solver, chapter 24), 1 s of run; `posts_128.vertex`. The README’s warnings about keeping `NFINEST` and the domain equal in the script and the input are chapter 8a’s rule.

CIB (1): RIGID BODIES WITH MOBILITY SOLVERS

"CIB Examples. An Immersed Boundary Method for Rigid Bodies. Bakytzhan Kallemov, Amneet Pal Singh Bhalla, Boyce E. Griffith, and Aleksandar Donev. This paper is on the google drive. Note that it is the more recent paper for two different methods used for rigid body simulations. Some of the examples considered in this paper appear in these CIB examples." (2020, slide 12)

`CIBMethod` with `CIBStaggeredStokesSolver`: the rigidity constraint written as a saddle-point problem and solved with a *mobility* matrix (the velocity a body gets from a force), which is why these examples ship a `petsc_options.dat` and keys like `STOKES_TOL`, `DELTA` ("regularization parameter for mobility matrix") and `VISC_CFL = 1E-16`. Three of the five are steady Stokes problems that take *one* time step: "this example only runs one DT" (slide 13) is by design.

Where it sits: chapter 14's `ConstraintIBMethod` corrects velocities and needs no linear solve of its own; `CIBMethod` solves for the constraint forces exactly, which is what a suspension of many rigid particles at zero Reynolds number needs, and what makes a single CIB step expensive. The paper: B. Kallemov, A. P. S. Bhalla, B. E. Griffith and A. Donev, *An immersed boundary method for rigid bodies*, Commun. Appl. Math. Comput. Sci. 11 (2016) 79–141.

CIB (2): THE FIVE EXAMPLES

ex0	"This example shows how to set various solvers for a fully prescribed kinematics for a 2d body." (README). Lecture: "basically a cylinder in 2D that rotated in a prescribed way. I only had two time steps that wrote to a visualization file, and checking the input file this example only runs one DT." $\rho = 0$, $\mu = 1$: Stokes.
ex1	"a mixture of prescribed and free DOFs for two bodies. The outer shell (structure 0) moves relative to the inner sphere (structure 1). The mobility matrices ... are formed independently and are distributed to different processors ... Forces and torques on both spheres (strct_id_0) and the inner sphere (strct_id_1) are computed via control volume analysis." 3D; one step. Lecture: "Stokes flow between two concentric spherical shells."
ex2	"A 2d plate is dragged with an imposed velocity for a certain time, after which it translates freely but with locked rotational motion." $\mathrm{Re} = 20$, 15 s; `input2d.plate.amr` adds a level. Lecture: "This example outputs many times. In the 2 hour debug queue I don't see rotation." (none is allowed: the rotation is locked.)
ex3	flow through a nozzle, $\mathrm{Re} = 10$, 100 steps; no README. Lecture: "The snapshot shows the velocity vectors and the color map shows the pressure. This example comes directly from the paper in the section on flow through a nozzle."
ex4	a falling sphere, 3D, $\rho = 2$, 50 s (`input3d.test.long`: 1.8 s); `check_sol.m`. Lecture: "The movement of the sphere is driven by gravity. I don't believe this example is in the paper, look at the code to see how it is handled."

CONSTRAINTIB: CHAPTER 14'S NINE, IN ONE LINE EACH

The constraint method of Bhalla et al. (2013), velocity correction without mobility matrices; the lecture spent six slides on them and chapter 14 ran three.

folder	dim	what	this series
`stokes_first_problem`	2D	a plate started impulsively (paper 7.1.1); prescribed	–
`moving_plate`	2D	a plate dragged across the fluid at $\mathrm{Re} = 20$ (7.1.2); prescribed, velocity by a C callback	–
`oscillating_rigid_cylinder`	2D	a disc oscillating at $\mathrm{Re} = 100$, $\mathrm{KC} = 5$ (7.1.4); prescribed	ConstraintIB/01
`flow_past_cylinder`	2D	a disc towed at constant speed; prescribed	–
`impulsively_started_cylinder`	2D	a disc given a body-force impulse; free (7.2.2)	ConstraintIB/02
`falling_sphere`	3D	a sphere sinking by a gravity body force in the solid region; free	–
`eel2d`	2D	a prescribed travelling wave, free swimming (7.3)	ConstraintIB/03
`eel3d`	3D	the same body in 3D (`eelgenerator3d.cpp`, `InterpolationCoefs.m`)	–
`knifefish`	3D	a knifefish with a ribbon fin (`KnifeFish.vertex`, 458k points, `RadiusAmp.dat`)	–

Slide 24 of the lecture: "Other examples include free swimming cases of eel swimming in 2D and 3D as well as a 3D knifefish. I have run the 2D eel, but the download is taking too long to show today. It is worth running the 3D eel and knifefish as well. There is also a falling sphere example of rigid body motion in 3D." The 3D eel and the knifefish are Puma jobs: $17{,}482$ and $457{,}936$ vertices.

NAVIER_STOKES (1): THE FLUID SOLVER ALONE

example	what it is (0.19.0)	the 2013 deck
ex0	an exact travelling-wave solution of Navier–Stokes in a periodic box (`VelocityInitialConditions`, `PressureInitialConditions`, `gcoef = U`), 10 steps; `input2d.forced_flow`: the same with a body force, 0.5 s; `input3d`	"cases that can be used to test convergence against analytical solutions. Ex0 uses an exact solution with traveling waves due to forced flow. Looking at the input files will show how this is done. *I have not run all N-S examples."
ex1	the same idea with a separate `convergence_tester.cpp` that compares two runs (chapter 16's grid study); $\mu = 10^{-4}$, three levels, 2 s	–
ex2	a lid-driven cavity at $\mathrm{Re} = 1000$, three levels from $N = 4, 100$ steps; `analyzer.m`, `L1norm.m`, `L2norm.m` compare with the Ghia benchmark	"This example is developing cavity flow. The left, right and bottom sides are no slip. The top has constant velocity in the x-direction and no flow in the y-direction. Warning, this outputs A LOT of data!"

These are test programs for the fluid solver, with no structure in the box; the solver classes are the ones every IB example creates (`INSStaggeredHierarchyIntegrator`, or `INSCollocatedHierarchyIntegrator` when the input selects it), so an input block that puzzles you in an IB example can be studied here on its own.

NAVIER_STOKES (2): CAVITIES AND CONVECTION

example	what it is (0.19.0)	the 2013 deck
ex3	the cavity again with `analyzer_256.m`: the 256-cell reference	–
ex4	the travelling-wave flow of ex0 with an advected–diffused scalar carried along (`AdvDiffSemiImplicitHierarchyIntegrator`), 10 steps; `input3d`	"This is an advection diffusion example. Velocity boundary and initial conditions for an flow field with an analytical solution are in input2. Warning, this outputs A LOT of data!"
ex5	Boussinesq convection: a temperature field with `KAPPA = 1e-6` and a buoyancy force `GAMMA = 1e2` (`BoussinesqForcing.cpp`), a 1×2 box, 128 cells, 100 steps	"This example contains and temperature gradient and uses the Boussinesq approximation. The left, right and bottom sides are no slip. The top has constant velocity . . . Warning, this outputs A LOT of data!" (three pictures of the plumes)
ex6	ex5 with thermal fluctuations (`INSStaggeredStochasticForcing`, `AdvDiffStochasticForcing`, `SEED`), $\mu = 10^{-3}$, 10 steps	–

"A LOT of data": `viz_dump_interval = 1` in the 2013 inputs. Today's inputs dump every few steps and run 10 or 100 steps; these are test programs, and chapter 16 uses ex1 and ex2 as the convergence examples.

ADVECT AND ADV_DIFF: A SCALAR IN A GIVEN FLOW

`advect` (one example, no sub-folders): “A blob of something with concentration Q (plotted) is advected in a given velocity field. It does not diffuse. Velocity field in input2D: `UFunction { init_type = "UNIFORM" uniform_u = -2.0,1.0 }`. It has a Gaussian distribution: `QInit { init_type = "GAUSSIAN" kappa = 0.001 }`” (2013, slide 17). `AdvectorExplicitPredictorPatchOps` alone, no fluid solver; `QInit.cpp`, `UFunction.cpp` set the blob and the velocity.

`adv_diff/ex0`: “In this example, the blob also diffuses. It is again in a prescribed flow field. See input2d” (slide 18); the same two helper classes, `AdvDiffSemiImplicitHierarchyIntegrator`, and the scripts `converg2d`, `converg3d` that run it at several resolutions (chapter 16). Both finish in a second.

ex1	the scalar carried by a Navier–Stokes flow with a forcing function; 0.125 s
ex2	`SOLVER_TYPE = "SEMI_IMPLICIT"`, `KAPPA = 1e-2`, a Dirichlet value `C` on one side; 2 s
ex3–ex7	the flux-based volume-penalization method for Neumann and Robin conditions on bodies given by a level set (a cylinder, a hexagram annulus, a 1D test, an egg and a torus, a circular annulus): Poisson problems solved in one step, with READMEs that name the manufactured solutions (Sakurai et al.; Thirumalaisamy, Patankar and Bhalla, JCP 448 (2022) 110726)
ex8	steady natural convection in a concentric annulus at $\mathrm{Ra} = 5 \times 10^4$, $\mathrm{Pr} = 0.7$, with the penalized boundaries; 10 s; writes a temperature profile (binary; `write_ascii.cpp` converts)

These are the scalar-transport machinery that chapter 24 couples to an IB structure (oxygen round a coral, heat round a wing); ex0 and ex2 are the ones to run first.

FREE SURFACES (1): LEVEL_SET AND MULTIPHASE_FLOW

`level_set` ex0, ex1	“A test program to demonstrate initialization (via fast-sweeping method) and advection of a level set distance function”; ex1 the same by relaxation, “to study the spatial convergence rate of the ENO subcell-fix method”. A circle in a box; seconds.
`multiphase_flow` ex0	“application of a surface tension force at the boundary of a drop in static equilibrium in an ambient fluid” (`SIGMA = 73`); 2D and 3D
ex1, ex2	“a circular cylinder impacting an air-water interface” (`RigidBodyKinematics`, `input2d_free_fall.cyl` half-buoyant); ex2 “uses finite element mesh for the solid body” (libMesh)
ex3, ex5, ex6	“the importance of consistent mass and momentum transport”: a density-10^6 droplet convected; a collapsing water column (dam break; 2D and 3D); a droplet splashing on a thin liquid layer
ex4	“a gravitational force for a stationary water-air tank and tank filling with water”

Air and water throughout: $\rho = 10^3$ and 1.2, $\mu = 10^{-3}$ and 1.8×10^{-5}, $g = 9.81$, with `GRAV_TYPE = "FLOW"` or `"FULL"` choosing how gravity enters; a level set marks the interface and `LSLocate*` classes place it.

FREE SURFACES (2): BODIES AT THE INTERFACE, `wave_tank`, `vc_navier_stokes`

ex7, ex8, ex9	a cylinder splashing into two fluids; a buoyant cylinder (sphere in 3D) heaving on an air-water interface; “static equilibrium of a stationary cylinder on an air-water interface and the differences between the ‘full’ and ‘flow’ gravitational forces”
ex10–ex13	water entry of a cylinder; a rectangular barge rolling on the interface; a half-buoyant wedge impacting the interface (2D and 3D); “a gravity-driven 3D water column impacting a stationary obstacle”. The vertex files of ex10–ex13 “automatically generated by the build process” from `*Gen.cpp`
`wave_tank` ex0, ex1	“the accuracy of Stokes first, second and fifth order waves in a numerical wave tank” (six inputs, MATLAB comparisons); “a second-order Stokes wave impacting a trapezoidal obstacle”
`vc_navier_stokes` ex0–ex3	“the variable-coefficient, incompressible Navier-Stokes solver yields the expected order of accuracy”: manufactured solutions with density and viscosity jumps of 10^3 and 10^2 (`ManufacturedSolution_2D.m`, `extract_error.sh`); ex2 residual convergence with a bubble; ex3 the inputs of a paper’s section 6.1

The 2020 lecture ended with “For next week, I’d like to go through the non-Newtonian and level set examples”; that week’s deck does not exist, and these two slides stand in for it. The multiphase examples with a body use `RigidBodyKinematics` classes of their own (not chapter 14’s) and a level set for the body too.

OTHER SOLIDS (1): IBLEVELSET AND IIM

`IBLevelSet` ex0–ex6	fully Eulerian FSI: the body is a level set, not points. ex0 “initialization of level set from STL mesh” (`CircleMesh.msh`); ex1 “A fully Eulerian FSI implementation of examples/ConstraintIB/oscillating_rigid_cylinder” (chapter 14’s ConstraintIB/01 done the other way, 26 s); ex2 a half-buoyant cylinder hitting water; ex3 “coupling of IBLevelSet and IBFE methods” (a rigid rod and an elastic structure); ex4, ex5 a self-propelling pitching airfoil, one and two; ex6 a floating block with a mass-preserving level set
`IIM` ex0–ex9	the immersed interface method and the immersed Lagrangian–Eulerian (ILE) method on libMesh surfaces (`.e` files): flow past a cylinder (“Sec. 4.5 of Kolahdouz et al., JCP 2020”) and a square; Hagen–Poiseuille pipe flow in 3D; a swimming thin airfoil; vortex-induced vibration of a disc; a falling sphere (glass, steel); the Turek–Hron tail; a soft disc in a cavity; a reservoir instability; a flexible plate in a flow phantom (3D)

`IBLevelSet/ex1` and `IIM/ex0` are the same cylinder as ConstraintIB/01 and IB/explicit/ex2 done by two other methods, and the comparison is instructive.

OTHER SOLIDS AND FLUIDS (2): DLM, IMP, COMPLEX_FLUIDS, FE_MECHANICS

`DLM` ex0	the Turek–Hron benchmark (a 2.46×0.41 m channel, $D = 0.1$ m, $\mathrm{Re} = 200$) with distributed Lagrange multipliers; five inputs vary the excess mass and the stiffness
`IMP/explicit` ex0	the immersed material point method: a `main.cpp` and an input, no structure files
`complex_fluids` ex0–ex4	Oldroyd-B fluids (`FLUID_MODEL = "OLDROYDB"`): pipe flow with an exact solution (error norms printed); a four-roll mill by a background force ("As the Weissenberg number increases, singularities in the trace form at extensional points"); flow past a confined cylinder with an IBFE structure and drag output; a lid-driven cavity with a quartic lid; the four-roll mill with immersed rollers
`fe_mechanics` ex0	Cook's membrane, solid mechanics without fluid, 2D and 3D

`complex_fluids` is the lecture's "non-Newtonian"; the rest are research examples that document their papers.

IBFE/explicit: twelve examples for Part IV

The finite-element IB examples, `ex0` to `ex11`, need libMesh (`REQUIRES IBAMR_HAVE_LIBMESH`; autoibamr builds it). They are toured in chapter 25 and used from chapter 17 on; here only the map (corrected 2026-10-15 from the sources; an earlier version of this slide listed this series' own examples under IBAMR's numbers): ex0 a pressurized elastic ring at rest, ex1 the same ring started as an ellipse, ex2 a soft plate across a box pushed by a traction difference, ex3 a soft block on the floor of a lid-driven cavity, ex4 a soft disc in a lid-driven cavity (2D and 3D), ex5 a tethered cylinder in a stream at $\mathrm{Re} = 200$ as a surface mesh or a solid (2D and 3D), ex6 the Turek–Hron beam behind a cylinder, ex7 a collapsible channel (chapter 23), ex8 a clamped beam over a slot loaded from below, ex9 a cylinder surface tethered to a prescribed position, ex10 a free rigid cylinder in Couette flow by direct forcing, ex11 flow past a fixed cylinder at $\mathrm{Re} = 200$ by direct forcing. Chapter 25 tours and runs them.

> The 2020 lecture's slide 2 called them "IBFE – Hybrid IB/Finite element examples. Hybrid finite difference/finite element immersed boundary method"; "Then I'd like to revisit the IBFE ones" (slide 25) became the IBFE lecture series of Part IV.

THE OLD NAMES AND THE NEW (1): THE 2013 AND 2015 DECKS' IB EXAMPLES

the 2013/2015 deck said	today (0.19.0)	notes
`IB/explicit/ex1` – Bouncing rubberband; ex1 – Bouncing shell	`IB/explicit/ex1` (band, shells)	ex0 (the pressurized band without files) is newer
`ex2` – Flow past cylinder (3D sphere/cylinder; targets without a `.target` file; time-dependent inflow)	`IB/explicit/ex2`	unchanged in substance; `compute_St.m` added
`ex3` – Flow past a fiber (mass "in an older way"; penalty IB)	`IB/explicit/ex3`	the `uniform_bdry_mass` keys are still the way; chapter 13's `.mass` file is the other
`ex4` – Kirchoff Rods (`.rod`, `.director`)	`IB/explicit/ex4`	3D only

THE OLD NAMES AND THE NEW (2): THE REST, AND THE DEAD LINKS

the 2013/2015 deck said	today (0.19.0)	notes
–	`ex5`, `ex6`, `ex7`	fluctuating hydrodynamics, the minimal integrator, the posts: after 2013
`navier_stokes/ex0, ex2, ex4, ex5`	the same numbers	ex1, ex3, ex6 added; “A LOT of data” no longer
`advect`; `adv_diff/ex0`	the same	`adv_diff` ex1–ex8 added
IB/staggered_grid/explicit (2012)	`IB/explicit`	the collocated-grid variants went; the staggered-grid solver is the one, and the inputs can still select the collocated one

Everything the 2013 deck said about ex1–ex4 and the four Navier–Stokes examples is still true of the current folders; its screenshots (an ellipse of vorticity, a shell of springs, a cylinder wake, a filament, the cavity, the travelling waves, the Boussinesq plumes, the blob) are what you will see. Its links to `math.nyu.edu` and `pof.aip.org` are dead; the papers are Zhu & Peskin, J. Comput. Phys. 179 (2002) 452–468, Kim & Peskin, Phys. Fluids 19 (2007) 053103, and Lim, Ferent, Wang & Peskin, Phys. Fluids 22 (2010) 024104 (the twisted-rod figure the deck reproduced).

WHAT WAS RUN FOR THIS CHAPTER (1): IB, CIB, NAVIER–STOKES

Each example copied, built with the plain `CMakeLists.txt` of slide 10 against the installed 0.19.0, and run on two ranks for at most 120 s (`timeout`); "exit 124" means the two minutes ran out with the example still going, which is a pass. Steps are what the log counted.

example	input	build	run	steps in ≤ 120 s	viz folder
`IB/explicit/ex0`	`input2d`	ok, 37 s	finished, 8 s	2200	`viz_IB2d`
`IB/explicit/ex1`	`input2d`	ok, 29 s	finished, 8 s	2200	`viz_IB2d`
`IB/explicit/ex2`	`input2d.cylinder`	ok, 29 s	running at 120 s	9649	`viz_IB2d`
`IB/explicit/ex3`	`input2d`	ok, 34 s	running at 120 s	10023	`viz_IB2d`
`IB/explicit/ex5`	`input2d`	ok, 29 s	finished, 2 s	100 (its own loop)	`viz_IB2d`
`IB/explicit/ex6`	`input2d`	ok, 30 s	finished, 3 s	2200	`viz_IB2d`
`IB/explicit/ex4`	`input3d`	ok, 31 s	running at 120 s	3147	`viz_IB3d`
`IB/explicit/ex7`	`input3d`	ok, 32 s	running at 120 s	4527	`viz_IB3d`
`CIB/ex0`	`input2d`	ok, 31 s	finished, 0 s	1 (Stokes)	`viz_cylinder2d`
`CIB/ex2`	`input2d.plate`	ok, 32 s	running at 120 s	199	`viz_2d`
`CIB/ex3`	`input2d.nozzle`	ok, 31 s	running at 120 s	33	`viz_nozzle2d`
`CIB/ex1`	`input3d`	ok, 32 s	finished, 60 s	1 (Stokes)	`viz_icosahedron3d`
`CIB/ex4`	`input3d.test.long`	ok, 31 s	running at 120 s	25	`viz_3d`
`navier_stokes/ex0`	`input2d`	ok, 36 s	finished, 0 s	10	`viz_INS2d`
`navier_stokes/ex1`	`input2d`	ok, 30 s	running at 120 s	1407	`viz_INS2d`
`navier_stokes/ex2`	`input2d`	ok, 54 s	finished, 2 s	100	`viz_INS2d`
`navier_stokes/ex3`	`input2d`	ok, 52 s	finished, 2 s	100	`viz_INS2d`
`navier_stokes/ex4`	`input2d`	ok, 67 s	finished, 0 s	10	`viz_INS2d`
`navier_stokes/ex5`	`input2d`	ok, 58 s	running at 120 s	2597	`viz_INS2d`
`navier_stokes/ex6`	`input2d`	ok, 43 s	finished, 1 s	10	`viz_INS2d`

WHAT WAS RUN FOR THIS CHAPTER (2): SCALARS, LEVEL SETS, FREE SURFACES, COMPLEX FLUIDS

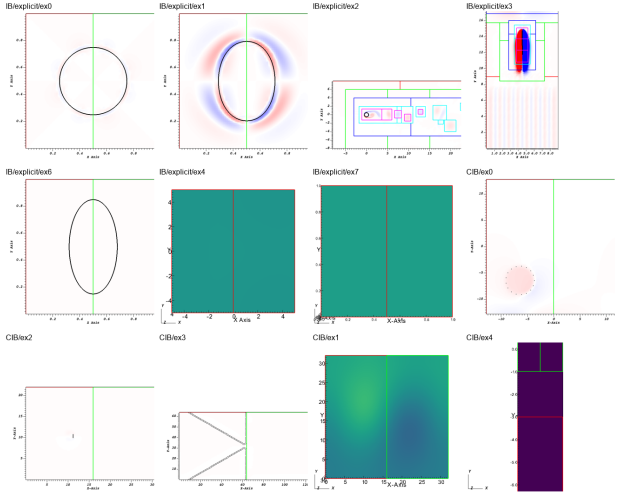
example	input	build	run	steps in ≤ 120 s	viz folder
`adv_diff/ex0`	`input2d`	ok, 35 s	finished, 1 s	132	`viz_adv_diff2d`
`adv_diff/ex1`	`input2d`	ok, 35 s	finished, 0 s	128	`viz_adv_diff2d`
`adv_diff/ex2`	`input2d`	ok, 35 s	finished, 0 s	512	`viz_adv_diff2d`
`adv_diff/ex3`	`input2d`	ok, 36 s	finished, 1 s	1 (a Poisson solve)	`viz_BP_Poisson2d`
`adv_diff/ex8`	`input2d`	ok, 38 s	running at 120 s	392	`viz_bp_free_convection_2d`
`advect`	`input2d`	ok, 42 s	finished, 1 s	33	`viz_advect2d`
`level_set/ex0`	`input2d`	ok, 38 s	finished, 1 s	65	`viz_advect2d`
`level_set/ex1`	`input2d`	ok, 38 s	finished, 0 s	none (a one-shot test)	`viz_advect2d`
`multiphase_flow/ex0`	`input2d`	ok, 36 s	running at 120 s	7972	`viz_INS2d`
`multiphase_flow/ex3`	`input2d`	ok, 36 s	running at 120 s	5253	`viz_INS2d`
`multiphase_flow/ex5`	`input2d`	ok, 36 s	running at 120 s	621	`viz_INS2d`
`complex_fluids/ex0`	`input2d`	ok, 42 s	finished, 42 s	12800	none written
`complex_fluids/ex1`	`input2d`	ok, 34 s	finished, 6 s	1000	none written
`complex_fluids/ex3`	`input2d`	ok, 33 s	running at 120 s	658	none written

WHAT WAS RUN FOR THIS CHAPTER (3): THE REMAINING SUITES, AND THE TALLY

example	input	build	run	steps in ≤ 120 s	viz folder
`IMP/explicit/ex0`	`input2d`	ok, 30 s	running at 120 s	7865	`viz_IB2d`
`IIM/ex0`	`input2d`	ok, 33 s	running at 120 s	4480	`viz_IIM_2d`
`vc_navier_stokes/ex0`	`input2d`	ok, 73 s	finished, 5 s	100	`viz_INS2d`
`wave_tank/ex0`	`input2d_CaseA`	ok, 67 s	running at 120 s	13	`viz_INS2d`
`fe_mechanics/ex0`	`input2d`	ok, 12 s	finished, 94 s	40000	none written
`DLM/ex0`	`input2d`	ok, 44 s	running at 120 s	6430	`viz_IB2d`
`IBLevelSet/ex0`	`input2d`	ok, 51 s	exit 255	0	--

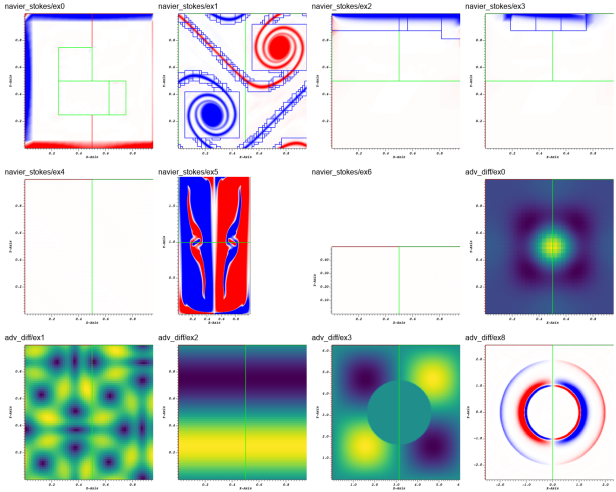
Of the 41 examples, 41 built with the six-line `CMakeLists.txt` (none failed to build); 22 finished their shipped run inside two minutes, 18 were still running when the two minutes ended, 1 stopped otherwise (`IBLevelSet/ex0`: exit 255). `IB/explicit/ex5`, `level_set/ex1` and `adv_diff/ex3` print no step banner, so their step counts are from their own output; `complex_fluids` ex0, ex1, ex3 and `fe_mechanics/ex0` write no visualization folder (they print norms or errors instead). The one failure, `IBLevelSet/ex0`, stopped at start-up with `Failed assertion: mesh->mesh_dimension() == mesh->spatial_dimension()` from `IBFEMethod`: its `CircleMesh.msh` is read by libMesh 1.7.8 as a one-dimensional curve where the example expects a two-dimensional mesh, a mesh-format question for chapter 18. Builds took 30 to 70 s each on 8 cores; IBFE and ConstraintIB were left to chapters 25 and 14.

GALLERY (1): WHAT THE RUNS LOOK LIKE



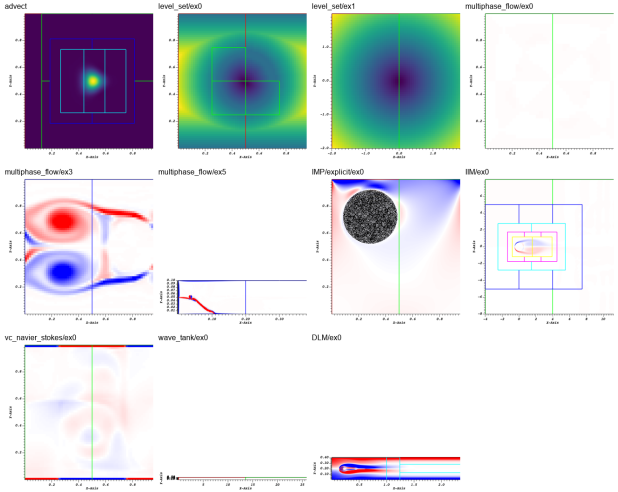
Last saved frame of each two-minute run, rendered headless with VisIt 3.5 (chapter 26): the first of Omega, P, Q, T or the level set, with the Lagrangian mesh where there is one. Panels: IB/explicit/ex0, IB/explicit/ex1, IB/explicit/ex2, IB/explicit/ex3, IB/explicit/ex6, IB/explicit/ex4, IB/explicit/ex7, CIB/ex0, CIB/ex2, CIB/ex3, CIB/ex1, CIB/ex4.

GALLERY (2)



Same rendering. Panels: navier_stokes/ex0, navier_stokes/ex1, navier_stokes/ex2, navier_stokes/ex3, navier_stokes/ex4, navier_stokes/ex5, navier_stokes/ex6, adv_diff/ex0, adv_diff/ex1, adv_diff/ex2, adv_diff/ex3, adv_diff/ex8.

GALLERY (3)



Same rendering. Panels: advect, level_set/ex0, level_set/ex1, multiphase_flow/ex0, multiphase_flow/ex3, multiphase_flow/ex5, IMP/explicit/ex0, IIM/ex0, vc_navier_stokes/ex0, wave_tank/ex0, DLM/ex0.

WHERE TO START, FOR A GIVEN PROBLEM

you want to simulate	start from	then read
an elastic body of fibres (a heart tube, a wing, a band)	`IB/explicit/ex1`, this series' IB/01	chapters 6, 9a, 9b, 10
a rigid body held in a flow	`IB/explicit/ex2` (tethers), IB/03, IB/04	chapters 7, 9a, 11
a flexible filament or flag with mass	`IB/explicit/ex3`, IB/12	chapters 10, 13
a rigid body moving as told, or free	`ConstraintIB`, ConstraintIB/01–03; `IBLevelSet/ex1`; `IIM/ex4`	chapter 14
a swimmer with a prescribed gait	`ConstraintIB/eel2d`, ConstraintIB/03	chapter 14
many rigid particles in Stokes flow	`CIB/ex1`, `ex4`	this chapter
a continuum elastic body (thick, nonlinear)	`IBFE/explicit/ex0`, `ex2`, IBFE/01	Part IV
a scalar (oxygen, heat, dye) with a structure	`adv_diff/ex0`, `ex2`; `IB/explicit/ex7`; IBFE/12	chapter 24
a free surface, a drop, a wave	`multiphase_flow/ex0`, `ex5`; `wave_tank/ex0`	this chapter
a viscoelastic fluid	`complex_fluids/ex0`, `ex1`	this chapter
the fluid solver's accuracy	`navier_stokes/ex0`, `ex1`; `adv_diff/ex0` with `converg2d`	chapter 16

Two rules. Copy the example nearest your problem and change one thing at a time (chapter 8a). And run it first as shipped, for its shipped time or two minutes, before you change anything: the gallery slides are what "as shipped" looks like.

WHAT CAN GO WRONG

what you see	why	fix
`Cannot find source file: example.cpp` or an unknown command `IBAMR_ADD_EXAMPLE`	you configured a copied example folder with its upstream `CMakeLists.txt`, which only works inside IBAMR's build	the plain `CMakeLists.txt` of slide 10
`undefined reference` to a generator's `main`, or two `main`s	`*Gen.cpp`, `convergence_tester.cpp`, `write_ascii.cpp` listed with `example.cpp`	leave them out, or make each its own target
`Example ... requires libMesh - skipping`	IBFE, IIM, `multiphase_flow/ex2`, `complex_fluids/ex2`, `IBLevelSet/ex3` need libMesh	autoibamr builds it; check `IBAMR_HAVE_LIBMESH` in the configure log
the run cannot open `cylinder2d.vertex`, `petsc_options.dat`, a `.e` mesh	you ran from the build folder; inputs are read relative to the working directory	run from the example folder with `./build/main2d`
the run never ends	`END_TIME = 300`, 10000, 400 s: these are cluster jobs	shorten `END_TIME`, or `timeout`; a Puma job for the real thing
the disk fills	`viz_dump_interval = 1` or restart dumps every few steps	raise the intervals (chapters 9b, 14)
"no flow fields" in VisIt	the Lagrangian database was opened alone	open `dumps.visit` too (chapter 26)
`input2c`	the 2012 slide's typo	`input2d`

Chapter 16 has the general ones (CFL aborts, parse errors, Puma limits, debugging).

HOMEWORK

1. Build and run `IB/explicit/ex1` with `input2d`, then with `input2d.shell` and `input2d.shell_circum_fibers`; describe in one sentence each how the three bands differ in VisIt (chapter 26).
2. Run `IB/explicit/ex0`, which has no `.vertex` file, and find in `example.cpp` the function that `IBRedundantInitializer` calls to make the points; change the number of points and the radius.
3. Run `IB/explicit/ex5` for 200 steps and watch the free points; then set `kT = 0` and say what happens to them, and why the lecture's description was wrong.
4. `navier_stokes/ex2`: run the cavity for 100 steps, then for $5{,}000$, and compare the centreline velocity with Ghia's values using the shipped `analyzer.m` (chapter 11 for the MATLAB side).
5. `adv_diff/ex0`: run `converg2d` (read it first; it runs the example at several resolutions) and report the order of accuracy you get (chapter 16).
6. Pick the example nearest your own problem from slide 40, run it as shipped for two minutes, and write down what you would change first.

RULES OF THUMB

- 98 examples in 18 suites; the IB ones are eight, and five of them run in seconds.
- Inside IBAMR's build: `make examples-IB`, `make IB-ex1`. Outside: copy the folder, drop in the plain `CMakeLists.txt`, link `IBAMR::IBAMR2d`.
- Run from the example folder; inputs are read from the working directory.
- Read `END_TIME` before running: seconds for most, steps for a few, cluster-long for many.
- A README, when there is one, names the paper; the input's first block names the physics.
- ex5 is fluctuating hydrodynamics; ex0 has no vertex file; ex6 is an integrator in two files.
- The 2013 deck's four IB examples and four Navier–Stokes examples are still there under the same numbers.
- Start from the nearest example and change one thing at a time.

SUMMARY

1. IBAMR 0.19.0 ships 98 examples in 18 suites: the standard IB method (8), rigid bodies three ways (CIB 5, ConstraintIB 9, DLM 1), finite elements (IBFE 12, IIM 10, IMP 1), fully Eulerian solids (IBLevelSet 7), the fluid solver alone (navier_stokes 7, vc_navier_stokes 4), scalars (adv_diff 9, advect 1), free surfaces (level_set 2, multiphase_flow 14, wave_tank 2), Oldroyd-B fluids (5) and solid mechanics alone (1).
2. They build inside IBAMR's own CMake tree as `examples-<suite>` and `<suite>-exN` targets, or outside it with a six-line `CMakeLists.txt` against an installed IBAMR; helper programs must be kept out of the main target.
3. The eight IB examples: a pressurized band made in code, a bouncing ellipse and shells, a tethered cylinder with a ramped inflow, a filament with mass in a gas, a twisted 3D rod, a cloud of points in a fluctuating fluid, the band under a two-file integrator, posts with a scalar. Four of them are the course's oldest examples and this series' IB/01–IB/04 descend from them.
4. Forty examples were built and started for this chapter; the tables and the gallery record what each did in two minutes, and which ones are cluster jobs.
5. Three statements of the older decks are corrected: ex5's purpose, the way ex0 gets its points, and the 2012 and 2020 build recipes, which belong to trees that no longer exist.

Chapter 16 closes Part III with what can go wrong, convergence tests and debugging; Part IV opens with IBFE.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The 2013/2015 deck (10 examples) and the 2020 lecture (16 examples plus ConstraintIB) are merged with the full 0.19.0 list, which is six times longer, and every suite gets at least a line.
- The build recipes of 2012 (`make examples` in a build tree that had `IB/staggered_grid`), 2013 and 2020 (a Makefile with `make.inc` and a cluster path) are replaced by the two CMake routes, with the upstream macro quoted so that the target names are not guesswork.
- The examples were run, not just described: forty builds and two-minute runs, with their outcomes in the deck, and VisIt pictures from those runs instead of the 2013 screenshots (which carried desktop windows and a user name).
- Corrections: ex5, ex0's `IBRedundantInitializer`, the dead links, “Kirchoff”, “input2c”, and the three “A LOT of data” warnings, which were about `viz_dump_interval = 1`.
- The map from problems to starting examples (slide 40) is new.

RECORDINGS (LISTED, NOT INCORPORATED)

The 2020 Zoom recording of IBAMR Lecture 6 (*Boyce's examples*) is in the course's Drive folder and listed in the front matter's video list (chapter 0.1). The 2013 deck was taught in MATH 769 without a recording. Nothing from the recording is reproduced here.

REFERENCES

- IBAMR 0.19.0, `examples/` and `examples/CMakeLists.txt`. https://github.com/IBAMR/IBAMR
- B. E. Griffith and S. Lim, *Simulating an elastic ring with bend and twist by an adaptive generalized immersed boundary method*, Commun. Comput. Phys. 12 (2012) 433–461 (IB/explicit/ex4).
- L. Zhu and C. S. Peskin, *Simulation of a flapping flexible filament in a flowing soap film by the immersed boundary method*, J. Comput. Phys. 179 (2002) 452–468; Y. Kim and C. S. Peskin, *Penalty immersed boundary method for an elastic boundary with mass*, Phys. Fluids 19 (2007) 053103 (ex3).
- B. Kallemov, A. P. S. Bhalla, B. E. Griffith and A. Donev, *An immersed boundary method for rigid bodies*, Commun. Appl. Math. Comput. Sci. 11 (2016) 79–141 (CIB); A. P. S. Bhalla, R. Bale, B. E. Griffith and N. A. Patankar, J. Comput. Phys. 250 (2013) 446–476 (ConstraintIB).
- R. Thirumalaisamy, N. A. Patankar and A. P. S. Bhalla, *Handling Neumann and Robin boundary conditions in a fictitious domain volume penalization framework*, J. Comput. Phys. 448 (2022) 110726 (adv_diff ex3–ex8); E. M. Kolahdouz et al., J. Comput. Phys. 2020 and 2021 (IIM).
- Sources: *IBAMR Lecture 6: What do Boyce's examples do?* (2020, 25 slides); *IBAMR Examples* (MATH 769, 2013, 18 slides; re-uploaded 2015 as tutorial 12); the 2012 lab ReadMe deck, slide 10; this series' `wsl/final/run_tour.sh` and `visit_tour.sh` (the forty runs and the gallery).